

EdgeHydra: Fault-Tolerant Edge Data Distribution Based on Erasure Coding

Qiang He , Senior Member, IEEE, Guobiao Zhang, Jiawei Wang, Ruikun Luo , Xiaohai Dai , Member, IEEE, Yuchong Hu , Feifei Chen , Senior Member, IEEE, Hai Jin , Fellow, IEEE, and Yun Yang , Senior Member, IEEE

Abstract—In the edge computing environment, app vendors can distribute popular data from the cloud to edge servers to provide low-latency data retrieval. A key problem is how to distribute these data from the cloud to edge servers cost-effectively. Under current schemes, a file is divided into some data blocks for parallel transmissions from the cloud to target edge servers. Edge servers can then combine received data blocks to reconstruct the file. While this method expedites the data distribution process, it presents potential drawbacks. It is sensitive to transmission delays and transmission failures caused by runtime exceptions like network fluctuations and server failures. This paper presents EdgeHydra, the first edge data distribution scheme that tackles this challenge through fault tolerance based on erasure coding. Under EdgeHydra, a file is encoded into data blocks and parity blocks for parallel transmission from the cloud to target edge servers. An edge server can reconstruct the file upon the receipt of a sufficient number of these blocks without having to wait for all the blocks in transmission. It also innovatively employs a leaderless block supplement mechanism to ensure the receipt of sufficient blocks for individual target edge servers. These improve the robustness of the data distribution process significantly. Extensive experiments show that EdgeHydra can tolerate delays and failures in individual transmission links effectively, outperforming the state-of-the-art scheme by up to 50.54% in distribution time.

Index Terms—Edge data caching, erasure coding, data distribution, distributed transmission.

I. INTRODUCTION

WITH the rapid development of modern technologies such as the 5G communication and artificial intelligence (AI), the number of smart devices and services has increased dramatically over the last decade [1], [2]. The requirements for real-time data processing have also increased profoundly [3], [4], [5]. In the traditional cloud computing paradigm, data is stored in remote data centers for user access. The distance between users and data centers introduces inevitable latency when users access these data [6], [7]. This significantly impacts the performance of applications [8], especially those requiring real-time data access [9].

Edge computing has emerged as a promising solution to this fundamental challenge by extending storage and computing resources from the cloud to the network edge, [10], [11], [12]. Based on users' data demands, popular data can be cached on edge servers by app vendors, enabling their users to access these data with ultralow latency [13], [14], [15].

An edge caching system serving a specific geographic region consists of a set of edge servers covering different geographical areas in the region [16]. Recently, a series of studies have focused on the design of edge caching strategies to meet app vendors' diverse needs [13], [17], [18], [19], [20]. To implement these caching strategies, app vendors must distribute data from cloud servers to targeted servers within the edge caching system. However, data transfer from the cloud to the edge takes time and incurs financial costs [21], [22]. Given a set of target edge servers in an edge caching system, denoted by $S = \{s_1, \dots, s_n\}$, edge data distribution (EDD) aims to transmit a file f from the cloud server to the system for caching on each of the edge servers in S . There are different ways to transmit f to S .

DataSync [23]: DataSync is an end-to-end data transfer service provided by Amazon. As shown in Fig. 1(a), f is transferred from the cloud server directly to target servers in a peer-to-peer manner. DataSync ensures low data distribution latency. However, considering that the cost of transferring data from the cloud to edge is much more than the cost of transferring data between edge servers (\$0.05-\$0.09 per GB versus \$0.01

Received 12 June 2024; revised 1 November 2024; accepted 4 November 2024. Date of publication 7 November 2024; date of current version 19 November 2024. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB4502400. Recommended for acceptance by R. Tolosana. (Corresponding author: Xiaohai Dai.)

Qiang He is with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China, and also with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: hqiang@hust.edu.cn).

Guobiao Zhang, Jiawei Wang, Ruikun Luo, Xiaohai Dai, and Hai Jin are with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: zhangguobiao@hust.edu.cn; kyriewang@hust.edu.cn; rkluo@hust.edu.cn; xhdai@hust.edu.cn; hjin@hust.edu.cn).

Yuchong Hu is with the Key Laboratory of Information Storage System Ministry of Education of China, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: yuchonghu@hust.edu.cn).

Feifei Chen is with the School of Information Technology, Deakin University, Geelong, VIC 3125, Australia (e-mail: feifei.chen@deakin.edu.au).

Yun Yang is with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: yyang@swin.edu.au).

Digital Object Identifier 10.1109/TPDS.2024.3493034

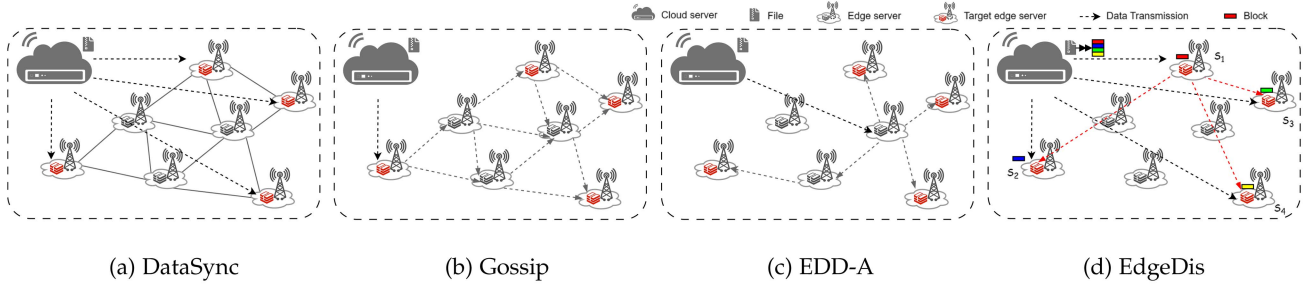


Fig. 1. Overview of exemplar distribution schemes. In (d), the data block transmissions from s_2 , s_3 and s_4 are omitted for figure readability.

per GB)^{1,2} DataSync-based edge data distribution is often not economic [24].

Gossip [25]: Gossip is widely used for data transfer across storage nodes in distributed systems [25]. To minimize distribution costs, only a single copy of the file is transmitted from the cloud to the edge caching system under the Gossip scheme. Based on Gossip, as shown in Fig. 1(b), f is first transferred to a selected entry server. Then, these edge servers transfer f process completes when all the target servers receive f . Compared with DataSync, the Gossip scheme reduces the transmission cost but requires gossiping among edge servers, which often causes substantial delays in the distribution process [21].

EDD-A [24]: EDD-A is a scheme specifically designed for edge data distribution, considering both transmission delay and cost. As shown in Fig. 1(c), EDD-A constructs a minimum Steiner tree with the cloud server serving as the root node, while target servers function as either leaf nodes or intermediate nodes. The cloud server then transfers f along the edges of the Steiner tree. EDD-A reduces the amount of data transferred from the cloud server to the edge caching system, lowering the transmission cost incurred. However, under the EDD-A scheme, the distribution of f is sensitive to runtime exceptions like network fluctuations and server failures [26]. A network delay or a server failure occurring over any path in the Steiner tree will impact the entire data distribution process significantly.

EdgeDis [21]: Edgedis is the state-of-the-art solution designed for the EDD problem. Under the Edgedis scheme, as shown in Fig. 1(d), the app vendor first slices f into some data blocks and transfers each of the blocks simultaneously to a selected entry server. These entry servers will then forward their block to the target servers, which can combine received blocks to reconstruct f . To improve the robustness of the EDD process, EdgeDis selects an edge server in the system as a coordinator to supply any missing blocks to target servers, ensuring that all the target servers will receive f eventually [21].

Challenges: Compared with DataSync, Gossip, and EDD-A, EdgeDis accelerates the EDD process with parallel block transmissions via entry servers. It also leverages a selected coordinator to ensure the completion of the EDD process when blocks are missing due to failed transmission from entry servers to target servers. However, EdgeDis assumes that entry servers will never fail or cause any delays in data transmissions. This

approach is not feasible in the real world, where edge servers often face runtime issues such as software exceptions, hardware faults, and cyberattacks [27], [28], [29]. These exceptions, as well as the dynamic workloads on edge servers, can undermine their responsiveness, causing delays and even disruptions to data transmissions going through them and consequently the entire EDD process. Take Fig. 1(d) as an example. Under Edgedis, the cloud server slices file f into some data blocks. It then sends one of these data blocks to each of a set of entry servers, i.e., $\{s_1, s_2, s_3, s_4\}$, which forward these data blocks to the target servers in the edge caching system. Due to network fluctuations or a sudden workload increase, data transmissions going through s_1 slow down substantially. When s_2 , s_3 , and s_4 have completed the transfer of their blocks to the target servers, s_1 is still transferring its data blocks. At this point, s_2 , s_3 , and s_4 must wait for s_1 to complete its transmission workloads before they can reconstruct f . The EDD is significantly delayed. In a worse case, when s_1 fails during an EDD process, the cloud must resend its block to another edge server for distribution. This delays the EDD process even worse, and in the meantime will incur extra transmission costs. In the remainder of this paper, entry servers that fail or experience transmission delays due to runtime exceptions are termed *faulty servers*, including *failed entry servers* and *slow entry servers*.

Our Solution: To mitigate the impacts caused by these faulty servers, this paper presents EdgeHydra, a new EDD scheme that distributes blocks encoded with erasure coding instead of data blocks. Under EdgeHydra, the cloud server first encodes file f into a set of coded blocks with erasure coding. Then, the cloud server sends these coded blocks to entry servers for distribution in the edge caching system. In this way, an edge server does not need all the coded blocks to reconstruct f . Instead, it can reconstruct f with only a proportion of these coded blocks. This allows EdgeHydra to tolerate slow or failed data transmission from faulty servers. It is the key idea of EdgeHydra.

Contributions: This paper makes the following main contributions.

- 1) EdgeHydra tolerates faulty servers during the EDD process with the application of erasure coding (EC), addressing a major limitation of existing EDD schemes. To our best knowledge, it is the first fault-tolerant EDD scheme.
- 2) It comes with a full set of enabling and supporting mechanisms specifically designed to facilitate EC-based fault-tolerant EDD.

¹<https://aws.amazon.com/ec2/pricing/on-demand/>

²<https://aws.amazon.com/s3/pricing/>

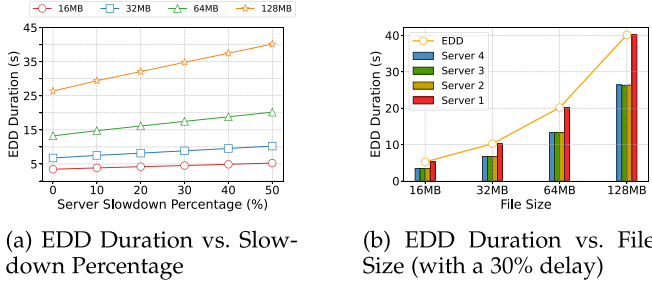


Fig. 2. Impact of slow entry servers.

- 3) EdgeHydra employs a leaderless block supplement mechanism to tolerate data transmission failures from entry servers to target servers, ensuring that target servers will always receive sufficient coded blocks to reconstruct the original file.
- 4) Extensive experiments are conducted on a testbed edge caching system, including 48 edge servers, to evaluate EdgeHydra against four competitive approaches, including three baselines and the state-of-the-art EDD scheme.

Paper Structure: The main structures are as follows. Section II provides a quantitative analysis to motivate this study research. Section III introduces EdgeHydra comprehensively. Section IV reports and analyzes the performance by extensive experiments. Section V summarizes the relevant literature, followed by the conclusion in Section VI.

II. MOTIVATION

Fig. 1(d) presents the process of distributing a file f under the EdgeDis scheme [21]:

- 1) File f is divided into some data blocks, four in this example. These data blocks are simultaneously transmitted from the cloud to four entry servers in the system.
- 2) Upon receiving a data block, each entry server acts as a sender, distributing the data block to other target servers within the system.
- 3) A coordinator is elected among the edge servers to oversee the completion of the EDD process. It checks and ensures that all edge servers have received all data blocks. In case any data blocks are missing due to failures, the coordinator organizes the retransmission of these blocks within the system to avoid extra costs and delays associated with re-fetching from the cloud.

Under the EdgeDis scheme, a target server needs all four data blocks to reconstruct f . The overall time taken to complete the EDD process depends on the last server that reconstructs f successfully. When slow or failed entry servers occur due to runtime exceptions, it can significantly delay the entire EDD process.

A. Impact of Slow Entry Servers

To quantitatively analyze the impact of slow entry servers, we implement EdgeDis on a testbed edge caching system including four target servers and set the data block size to 256KB. Fig. 2(a)

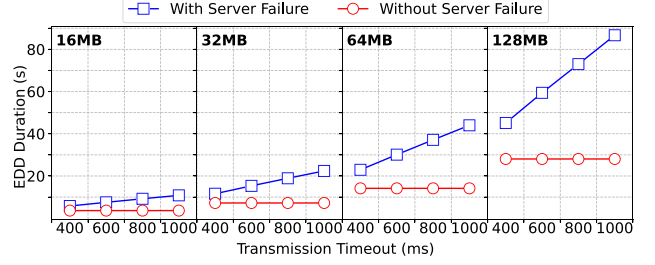


Fig. 3. Impact of transmission timeout.

illustrates the average time EdgeHydra takes to distribute a file of different sizes when there is a delay in data transmissions going through one of the entry servers. We can see that the delay slows down the entire EDD process significantly. When the slow server takes 50% more time than usual to complete data transmissions, the EDD process experiences a corresponding approximate 50% increase in delay.

To investigate the cause, we record the time each server took to transmit its data block during the EDD process and compare it against the time for EdgeDis to complete the entire EDD process. Fig. 2(b) shows the results. We can clearly see that while s_2, s_3 and s_4 take almost the same time to complete their block distribution tasks, s_1 takes much more time to complete its task and delays the entire EDD process tremendously. When the size of f is large, the delay caused by s_1 is prohibitively long, reaching up to 34.33% of the duration of the EDD.

B. Impact of Failed Entry Servers

Entry server failures will also delay an EDD process. Should it happen, EdgeDis sends corresponding data blocks to other entry servers for distribution based on a simple timeout-based mechanism. However, the timeout-based data block retransmission mechanism may also introduce undesirable delays to the EDD process. The cloud server sends the data block handled by a failed entry server to another entry server for distribution when the transmission timeout elapses. It is challenging to set a proper transmission timeout in the dynamic edge computing environment. If it is too long, the cloud server must wait a substantial time before transmitting the data block to another entry server for distribution. If the transmission timeout is overly short, data block re-transmissions can be easily triggered by minor network fluctuations or server slowdowns, leading to unnecessarily redundant data block transmissions.

To validate this, we experiment on EdgeDis to explore the impact of timeout settings on EDD duration under EdgeDis upon entry server failures. Fig. 3 summarises the results. As depicted, a longer timeout increases the EDD duration remarkably. A longer timeout takes the cloud server more time to realize entry server failures and begin data block re-transmission. Data block re-transmissions also take time. As demonstrated in the figure, when distributing a large file, a longer transmission timeout introduces a more significant delay in the EDD process, taking EdgeDis over 80 seconds to complete the EDD process with a

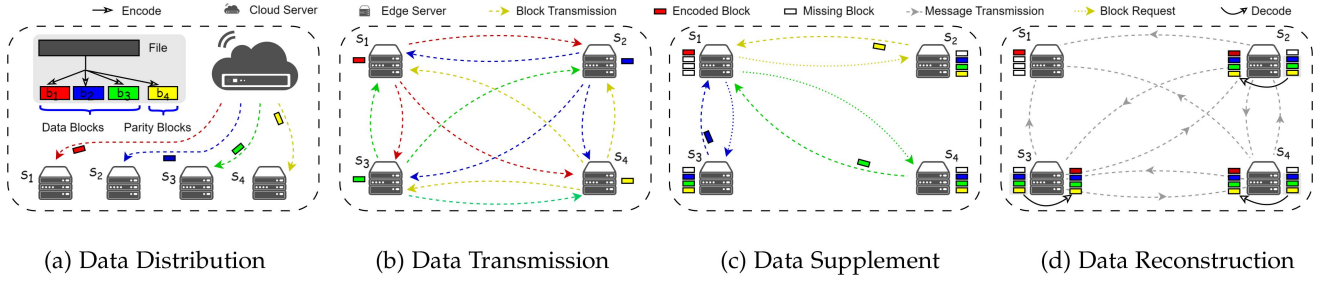


Fig. 4. Overview of EdgeHydra's data distribution process.

TABLE I
SUMMARY OF NOTATIONS

Notation	Description
b_i	i th coded block
cs	cloud server
f	file
k	number of data blocks through coding
m	number of parity blocks through coding
mes_{bd}	block distribution message
mes_{bdc}	block distribution completion message
mes_{bt}	block transmission message
mes_{brc}	block receipt message
mes_{brq}	block request message
mes_{bs}	block supplement message
mes_{dr}	data reconstruction message
mes_{hb}	heartbeat message
n	number of target server
s_i	i th target server

1,000 ms transmission timeout. This violates the requirement for fast system responses in the edge computing environment.

C. EdgeHydra Solution

We can see that slow and failed entry servers can delay the entire EDD process significantly. EdgeHydra tackles this challenge by transmitting files encoded with erasure coding. In this way, target servers do not have to receive all the data transmitted through every entry server. They can reconstruct the original file from part of the coded data. In this way, EdgeHydra minimizes the impact of slow and failed entry servers.

III. EDGEHYDRA DATA DISTRIBUTION

In this section, we introduce EdgeHydra and discuss how it mitigates delays in an EDD process by combating the faulty servers discussed above based on erasure coding. Table I summarizes the notations used in this paper.

Erasure Coding: As a widely used technique in storage systems, erasure coding can offer high data availability with relatively low storage overheads [30], [31], [32], [33]. Under an erasure coding scheme, a file f is first sliced into k data blocks, followed by the generation of m parity blocks through a coding process. These coded blocks (data blocks and parity blocks) are often distributed across multiple nodes in the storage system. Anyone can reconstruct f from any k of the total $k+m$ coded blocks. This is called a (k, m) erasure coding scheme. It provides

fault tolerance, allowing the system to tolerate the loss of up to m coded blocks. This inspired the design of EdgeHydra. It leverages erasure coding to address the robustness issues during the EDD process, especially those undermining the usefulness and performance of the state-of-the-art EDD scheme, i.e., faulty servers caused by runtime exceptions.

A. EdgeHydra Overview

Fig. 4 illustrates the four-stage procedure for distributing a file f to four target servers $s_1, \dots, s_4$ under EdgeHydra where $(k = 3, m = 1)$ erasure coding is employed to encode f .

- 1) In Stage 1, the cloud server encodes f into four coded blocks, i.e., three data blocks and a parity block. Then, it sends these coded blocks to $s_1, \dots, s_4$ for distribution, one each.
- 2) In Stage 2, $s_1, \dots, s_4$ transfer received coded blocks to each other. If an edge server receives sufficient coded data blocks, it proceeds to Stage 4.
- 3) In Stage 3, if a server has not received sufficient coded data blocks for file f reconstruction in Stage 2, it requests missing coded blocks from other edge servers. Upon receiving these coded blocks, it proceeds to Stage 4.
- 4) In Stage 4, edge servers reconstruct f from received coded blocks and notify all the entry servers to confirm the receipt of f . When all four target servers complete reconstructing f , the EDD process completes.

Note that after Stage 1, the distributions of individual coded blocks are carried out through Steps 2–4 asynchronously. The distribution of one coded block is not dependent on the distribution of any other coded blocks. Servers exchange messages to coordinate block transmission, which introduces a small amount of additional coordination overheads. It is evaluated experimentally in Section IV-B6.

B. Stage 1: Data Distribution

As illustrated in Fig. 4(a), to distribute a file f to n target servers, the cloud server cs , operated by the app vendor, encodes f into n coded blocks. Compared with transferring f as a whole - as in DataSync, Gossip, and EDD-A [24] - this approach allows cs to accelerate the EDD process via parallel data transmissions. It also reduces the data required for retransmission in case of transmission failures. The cs performs (k, m) encoding on f , where $n = k + m$, to produce $(k + m)$ coded blocks, denoted

as $b_1, \dots, b_n$. Next, cs sends these n coded blocks to n target servers for distribution, e.g., b_1 to s_1 , b_2 to s_2 , etc. Each target server, as an entry server, is responsible for transferring one coded block to other target servers in the system. When encoding f , cs obtains a sequential integer $blockID$ for each coded block to track their transmissions (Section III-C - Section III-E).

Under EdgeDis [21], cs slices f into n data blocks that are transferred to entry servers for distribution. Every target server must receive all these n blocks to reconstruct f . There is no room for errors. Now, under EdgeHydra, each target server can reconstruct file f from any k coded blocks. When a runtime exception slows down or shuts down one of the entry servers, the other edge servers will still be able to receive k coded blocks. In Fig. 4(a), cs employs ($k = 3, m = 1$) encoding, enabling EdgeHydra to tolerate one faulty server. A larger m will increase its robustness. The corresponding performance and overheads are evaluated in Section IV-B5.

When an entry server completes the distribution of a coded block, it will notify cs . After sending coded blocks to entry servers, cs employs an array to track their distribution, i.e., $blockStatus[]$ for tracking the transmissions of coded blocks. The variable is updated based on the updates from entry servers. For instance, $cs.blockStatus[i] = 1$ signifies that the system has successfully received the i th coded block. From the perspective of cs , the distribution of f completes when any k coded blocks are successfully transmitted to the system. That is also when cs can begin the distribution of the next file while the edge servers will complete the distribution of f within the system. This enables asynchronous distributions of files between Stage 1 and Stages 2–4.

To tolerate transmission failures in this stage, cs sets a timer with a distribution timeout for the distribution of coded blocks. Before the timer elapses, cs needs to confirm that at least k coded blocks have been successfully distributed to the system because these coded blocks will eventually guarantee the reconstruction of f for the target servers. Otherwise, cs needs to transmit the failed coded blocks to new entry servers for distribution when the timer elapses. The timer can be set up based on the block size. In general, a larger block size needs a longer timeout to avoid unnecessary re-transmission of coded blocks.

Workflow: The detailed workflow in this stage is as follows.

- **Step 1. Block Distribution Message Generation:** When transmitting an coded block b_i to entry server s_p , cs needs to encapsulate b_i , its $blockID$, and the corresponding erasure coding information $ecInfo$ into the block distribution message $mes_{bd}(b_i)$.
- **Step 2. Block Distribution Completion Message Receipt:** When s_p completes the transmission of b_i , it responds to cs with a block distribution completion message $mes_{bdc}(b_i)$ to notify cs and cs will update $cs.blockStatus[i] = 1$. When cs receives k different notifications from entry servers, the distribution of f is completed.

Remark: Erasure coding information, denoted as $ecInfo$, is included in the messages for transmitting a coded block b_i for target servers to facilitate the reconstruction of f , i.e., the $blockID$ of the first coded block ($beginID$) and the $blockID$ of the last coded block ($endID$) produced from f .

Pseudocode 1: Data Distribution.

```

1: // Stage 1 Step 1: Block Distribution Message
   Generation in  $cs$ 
2: When transmitting coded block  $b_i$  to  $s_p$  do
3:    $cs$  generates a block distribution message  $mes_{bd}(b_i)$ 
4:    $mes_{bd}.block = b_i$ 
5:    $mes_{bd}.blockID = i$ 
6:    $mes_{bd}.ecInfo = ecInfo(b_i)$ 
7:    $cs$  sends  $mes_{bd}(b_i)$  to  $s_p$ 
8: // Stage 1 Step 2: Block Distribution Completion
   Message Receipt in  $cs$ 
9: Upon receiving  $mes_{bdc}(b_i)$  from  $s_p$  do
10:   $cs.blockStatus[i] = 1$ 
11:  if received  $k$  different  $mes_{bdc}$  then
12:     $cs$  considers that the distribution of  $f$  completes

```

C. Stage 2: Data Transmission

Upon receiving block distribution message $mes_{bd}(b_i)$ from cs , entry server s_p generates a block transmission message $mes_{bt}(b_i)$ and distributes b_i . For instance, as shown in Fig. 4(b), s_1 is sending b_1 to other target servers. To ensure that all the target servers will eventually receive b_i , s_p employs an array $tranStatus[]$ to track the transmission of b_i . For example, $s_p.tranStatus[i, q] = 1$ signifies that the q th target server has successfully received coded block b_i . Every target server employs an array $blockStatus[]$ to record its receipts of coded blocks. If target server s_q has received coded block b_i , it sets $s_q.blockStatus[i] = 1$ and notifies s_p .

Similar to Raft [34], EdgeHydra aims to tolerate up to $\lfloor (n-1)/2 \rfloor$ server failures in an edge caching system with n edge servers. When s_p receives notifications from more than half of the target servers confirming their receipts of b_i , b_i can eventually be delivered to all target servers through the block supplement mechanism in Stage 3 (Section III-D). Thus, s_p notifies cs that the distribution of b_i is completed. Otherwise, cs thinks that the edge caching system has not received b_i successfully. If no more than k coded blocks have been successfully transmitted to the edge caching system, cs will retransmit b_i (Section III-B). Either way, EdgeHydra guarantees that f will eventually be distributed successfully, with or without b_i in the system.

Workflow: EdgeHydra employs the workflow below to transmit coded blocks within the system. The pseudocode is resented in Pseudocode 2.

- **Step 1. Block Transmission Message Generation:** Entry server s_p encapsulates b_i , its $blockID$, and the erasure coding information $ecInfo$ into a block transmission message $mes_{bt}(b_i)$, and transmits it to other target servers.
- **Step 2. Block Receipt Message Generation:** When a target server s_q receives $mes_{bt}(b_i)$ from entry server s_p , it first saves b_i in its local cache and sets $s_q.blockStatus[i] = 1$. Then, it generates a block receipt message $mes_{brc}(b_i)$ to inform s_p that it has received b_i . After that, it counts the number of received coded blocks for reconstructing f that produced b_i based on $mes_{bt}(b_i).ecInfo.beginID$

Pseudocode 2: Data Transmission.

```

1: // Stage 2 Step 1: Block Transmission Message
   Generation in  $s_p$ 
2: Upon receiving  $mes_{bd}(b_i)$  from  $cs$  do
3:    $s_p$  generates a block transmission message
      $mes_{bt}(b_i)$ 
4:    $mes_{bt}.block = b_i$ 
5:    $mes_{bt}.blockID = i$ 
6:    $mes_{bt}.ecInfo = mes_{bd}.ecInfo$ 
7:    $s_p$  sends  $mes_{bt}(b_i)$  to the other target servers
8: // Stage 2 Step 2: Block Receipt Message Generation in
    $s_q$ 
9: Upon receiving  $mes_{bt}(b_i)$  from  $s_p$  do
10:   $s_q$  stores the coded block  $b_i$  from  $mes_{bt}$ 
11:   $s_q.blockStatus[i] = 1$ 
12:   $s_q$  generates a block receipt message  $mes_{brc}$ 
13:   $mes_{brc}.serverID = q$ 
14:   $s_q$  sends  $mes_{brc}$  to  $s_p$  to confirm
15:   $s_q$  retrieves  $mes_{bt}.ecInfo$  for received coded blocks
16:  if missing coded blocks  $\leq mes_{bt}.ecInfo.k$  then
17:     $s_q$  reconstructs file  $f$ 
18: // Stage 2 Step 3: Block Transmission Status Inspection
   in  $s_p$ 
19: Upon receiving  $mes_{brc}(b_i)$  from  $s_q$  do
20:   $s_p.transStatus[i, q] = 1$ 
21:  if received  $\lceil (n+1)/2 \rceil - 1$  confirmation for  $b_i$  then
22:     $s_p$  responds to  $cs$  with a block distribution
     completion message for  $b_i$ 

```

and $mes_{bt}(b_i).ecInfo.endID$. If it has received k coded blocks or more, it proceeds to Stage 4 (Section III-E) to reconstruct f .

- **Step 3. Block Transmission Status Inspection:** When entry server s_p receives $mes_{brc}(b_i)$ from target server s_q , s_p will update $s_p.transStatus[i, q] = 1$, indicating that b_i has been transmitted to s_q successfully. If more than half of the target servers have confirmed their receipts of b_i , s_p notifies cs with a block distribution completion message (Section III-B).

Remark: Under EdgeHydra, target server s_q does not need coded block b_i from entry server s_p when it has reconstructed the file f from other k coded blocks. In such a case, s_q will send a data reconstruction message mes_{dr} to s_p . And s_p can immediately set $s_p.transStatus[i, q] = 1$ because s_q no longer needs b_i . This avoids unnecessary transmission of b_i to s_q . And s_p can notify cs of the successful distribution of b_i as soon as possible without delaying the distribution of the next file from cs . This represents an additional advantage offered by EdgeHydra's use of erasure coding.

D. Stage 3: Data Supplement

Distributing coded blocks instead of data blocks enables EdgeHydra to tolerate the distribution of m coded blocks into the edge caching system. However, in Stage 2, an entry server may successfully transmit its coded blocks to certain target servers but fails to others due to transmission failures. EdgeDis [21]

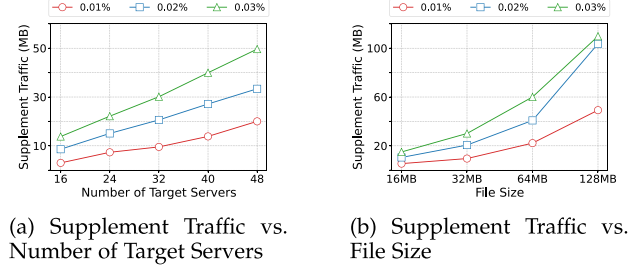


Fig. 5. Traffic of coordinator supplement.

selects a target server as a coordinator based on a Raft-like [34] leader election mechanism to address this issue. The coordinator collects data blocks from other target servers and supplies missing data blocks to target servers. However, having a single server collecting and re-transmitting all the missing data blocks places tremendous workloads on that server and may exhaust it, given edge servers' constrained compute and network capacities. In addition, under EdgeDis, the collection and re-transmission of data blocks require two rounds of data block transmissions. This can delay the EDD process substantially and incur excessive transmission costs.

To validate this numerically, we perform an experiment on EdgeDis to quantify the traffic incurred by the coordinator when supplying missing data in various EDD scenarios. In these scenarios, we vary three parameters: transmission failure rate, number of target servers, and file size. Fig. 5 reports the experimental results. It is evident that the coordinator experienced a substantial transmission workload at a 3% transmission failure rate, incurring additional traffic as much as 97.27% of the file size on average. As the number of target servers and the file size increased, the coordinator's supplement traffic also escalated. This indicates that when distributing large files to large edge caching systems under EdgeDis, the coordinator is likely to suffer from excessive transmission workloads, making it a potential performance bottleneck in the EDD process. In the meantime, a higher transmission failure rate also increases the extra traffic overheads significantly.

To overcome the above limitations, EdgeHydra employs a leaderless mechanism to supply missing coded blocks. As briefly illustrated in Fig. 4(c), under EdgeHydra, each target server periodically sends heartbeat messages to update other target servers on coded blocks it has received. Target servers missing coded blocks randomly request these coded blocks from a target server that possesses them. This distributes the supplement workloads across multiple servers, easing the burden on their resources. In the meantime, missing coded blocks are supplied to edge servers in need in parallel, minimizing the potential delays caused in the EDD process. Furthermore, it removes the possibility of single-point failure caused by the coordinator being overwhelmed by workloads or attacks under EdgeDis. Broadcasting heartbeat messages introduce traffic overheads. Fortunately, it constitutes only a tiny proportion of the overall traffic generated during the EDD process. A detailed analysis can be found in Section IV-B6.

Workflow: As shown in Pseudocode 3, the workflow in this stage is as follows.

- **Step 1. Heartbeat Broadcast:** The servers periodically broadcast heartbeat messages in the system to inform them of the coded blocks it has received. The message contains *serverID*, i.e., the ID of the server, and *receivedBlockIDs*, the IDs of the coded blocks it has received.
- **Step 2. Block Request:** Under EdgeHydra, a target server s_q can retrieve missing coded blocks from other target servers to ensure it has sufficient coded blocks for reconstructing file f . Upon receiving a heartbeat from s_h with coded block b_i , s_q checks whether it has already received b_i . If not, it sends a block request $mes_{brq}(b_i)$ to s_h for b_i . In the meantime, it sets a timer with a block transmission timeout and will not ask another server for b_i until the timer elapses. This block transmission timeout is set up based on the block size and the network bandwidth in the system. Typically, it can be set up as twice the diameter of the network connecting the edge servers. The maximum time it takes for an edge server to receive a coded block from another edge server in the system depends on the diameter of the network connecting them. Setting the timeout to twice the network diameter allows s_q to fetch a coded block delayed by temporary network congestion within the timeout while preventing edge servers from sending excessive block requests in a short period. If there are message errors, s_q will send a block request to a new server that holds b_i when the timeout elapses. This ensures the completion of the supplement of b_i .
- **Step 3. Block Supplement:** Upon receiving a block request $mes_{brq}(b_i)$ from s_q , the server s_h will generate a message $mes_{bs}(b_i)$ containing b_i , its *blockID* and the corresponding erasure coding information *ecInfo*. Then, $mes_{bs}(b_i)$ is transferred to s_q for block supplement. Upon receiving $mes_{bs}(b_i)$, s_q stores b_i and sets $s_q.blockStatus[i] = 1$. After that, similar to Step 2 in Stage 2 (Section III-C), s_q checks whether it can reconstruct f .

E. Stage 4: Data Reconstruction

When a target server s_q completes reconstructing file f , it sends a data reconstruction message mes_{dr} to all the entry servers, telling them it does not need any more coded blocks produced from f . As discussed above in Stage 2, this will stop entry servers from sending other coded blocks to s_q , accelerating the distribution process and saving on distribution costs. In the meantime, s_q can continue sending coded blocks to other target servers when it receives new coded blocks from the cloud or responds to block requests from other target servers.

Workflow: The detailed workflow in this stage is as follows, with pseudocode presented in Pseudocode 4.

- **Step 1. Data Reconstruction Message Generation:** When target server s_q reconstructs f successfully, it generates a data reconstruction message mes_{dr} and sends it to all the entry servers.
- **Step 2. Entry Server Quick Confirmation:** Upon receiving mes_{dr} from s_q , an entry server s_p will stop sending s_q

Pseudocode 3: Data Supplement.

```

1: // Stage 3 Step 1: Heartbeat Message  $mes_{hb}$  Generation
   in  $s_h$ 
2: Loop
3:    $s_h$  creates receivedBlockIDs to store the id of
     each coded block
4:    $s_h$  generates new heartbeat message  $mes_{hb}$ 
5:    $mes_{hb}.receivedBlockIDs = receivedBlockIDs$ 
6:    $mes_{hb}.ServerID = h$ 
7:    $s_h$  broadcasts  $mes_{hb}$ 
8: // Stage 3 Step 2: Heartbeat Handle and Missing coded
   blocks Request in  $s_q$ 
9: Upon receiving  $mes_{hb}$  from  $s_h$  do
10:  for  $i$  in  $mes_{hb}.receivedBlockIDs$  do
11:    if  $s_q$  misses  $b_i$  then
12:       $s_q$  generates a block request message  $mes_{brq}$ 
13:       $mes_{brq}.serverID = q$ 
14:       $mes_{brq}.blockID = i$ 
15:       $s_q$  sends  $mes_{brq}$  to server  $s_h$ 
16: // Stage 3 Step 3: Block Supplement in  $s_h$ 
17: Upon receiving  $mes_{brq}(b_i)$  from  $s_q$  do
18:    $s_h$  generates a block supplement message  $mes_{bs}$ 
19:    $mes_{bs}.block = b_i$ 
20:    $mes_{bs}.blockID = i$ 
21:    $mes_{bs}.ecInfo = b_i's ecInfo$ 
22:    $s_h$  sends  $mes_{bs}$  to  $s_q$ 

```

Pseudocode 4: Data Reconstruction.

```

1: // Stage 4 Step 1: Data Reconstruction Message
   Generation in  $s_q$ 
2: Upon reconstructing file  $f$  do
3:    $s_q$  generates a data reconstruction message  $mes_{dr}$ 
4:    $mes_{dr}.serverID = q$ 
5:    $s_q$  broadcasts  $mes_{dr}$  to confirm
6: // Stage 4 Step 2: Data Reconstruction Message Handle
   in  $s_p$ 
7: Upon receiving  $mes_{dr}$  from  $s_q$  do
8:   for  $blockID$  from  $beginID$  to  $endID$  do
9:      $s_p.transStatus[blockID, q] = 1$ 

```

other coded blocks produced from f . In the meantime, it will set $s_p.transStatus[blockID, q] = 1, \forall blockID \in \{beginID, \dots, endID\}$. In this way, s_p can confirm with the cloud server cs that it has completed the distribution of f as soon as possible.

IV. EXPERIMENTAL EVALUATION

We implement a prototype of EdgeHydra in Python 3.8 to conduct evaluations. It is architecturally composed of a cloud module and multiple edge modules. The cloud module runs on an Amazon EC2 instance. It implements the erasure coding algorithm and sends coded blocks to the edge modules on the edge servers in an edge caching system for distribution. Each

edge server runs an edge module that interacts with each other as well as the cloud module to implement EdgeHydra.

A. Experimental Settings

The testbed is comprised of a cloud server deployed within Amazon AWS and 48 edge servers deployed in a local data center. The cloud server is a c1m2.2xlarge virtual machine (VM) running Ubuntu 16.04 on 8 vCPU, 16 GB RAM, and 2 Gbps of network bandwidth. Each edge server runs Ubuntu 16.04 on 2 vCPU, 2 GB RAM, and 2 Gbps of network bandwidth. Deployed in the same local data center, the network latency between these edge servers is as low as 10 ms. The network latency between the cloud server and these edge servers ranges from 100 milliseconds to 150 milliseconds. EdgeHydra can distribute different types of data of varying sizes. In the experiment, EdgeHydra distributes video files. Popular video apps usually impose size limits on videos. For example, TikTok allows Android users to post videos up to 72 MB in size. To evaluate the applicability of EdgeHydra, we distribute data sized 1–144 MB, which is twice the size of the video files permitted on TikTok.

1) *Benchmark Approaches*: We conduct experiments to evaluate how EdgeHydra performs compared to three representative EDD methods and one state-of-the-art EDD method.

- *DataSync* [23]: This approach transmits file f from the cloud to all target servers simultaneously to complete the EDD process.
- *Gossip* [25]: This approach initially transfers f to an entry server in the edge caching system. Then, every 10 milliseconds, this server transmits f to two random edge servers. These servers, in turn, continue this process by periodically transmitting f to random servers until all target servers have received f . If the initial entry server fails, Gossip randomly selects another server to act as the new entry server and restarts the EDD process.
- *EDD-A* [24]: This approach identifies a minimum Steiner tree spanning from the cloud to all target servers and transmits f along paths in this tree. Should a node on one of the paths fail, EDD-A will generate a new minimum Steiner tree that excludes the failed node for the re-distribution of f .
- *EdgeDis* [21]: This state-of-the-art approach partitions f into some data blocks and transmits one of them to each target server. These target servers, as entry servers, transmit these blocks to other target servers for the reconstruction of f . In the meantime, a coordinator is elected in the system to collect data blocks and supply missing data blocks to target servers for reconstructing f .

2) *Performance Metrics*: EDD aims to distribute data from the cloud to all target servers cost-effectively and efficiently [16]. To evaluate the capability of EdgeHydra to achieve this objective within a dynamic edge environment, we employ two metrics for performance evaluation.

- *Distribution Time*: The distribution time, measured in seconds, represents the time required to complete data distribution. It is defined as the duration from the start of

TABLE II
PARAMETER SETTINGS

Parameter	Value	Default
n	16, 24, 32, 40, 48	32
fs	[1, 24]MB, [25, 48]MB, [49, 72]MB [73, 96]MB, [97, 120]MB, [121, 144]MB	[25, 48]MB
sn	1, 2, 3, 4, 5	3

the distribution to the point when the cloud server confirms the completion of the EDD process. The time for file reconstruction is multi-order magnitude shorter than the distribution time and thus is not included as part of the distribution time in the evaluation.

- *Distribution Cost*: The distribution cost is the economic cost incurred by the traffic generated during the EDD process. As introduced in Section I, data transmission from cloud to edge is about 5x - 9x more expensive than between edge-to-edge transmission. Accordingly, in the experiments, we set the cost for transmitting 1 MB of data from cloud to edge as one price unit, and the cost for transmitting the same amount of data between edge servers at $\frac{1}{9}$ units.

3) *Parameter Settings*: Edge servers broadcast heartbeats every 100 milliseconds within the system to notify each other of their received coded blocks (Section III-D). It is an order of magnitude longer than the network latency between two adjacent edge servers. This enables prompt notifications without excessive communication overheads.

To evaluate the performance of EdgeHydra comprehensively, we create various EDD scenarios by varying three parameters as summarized in Table II. For each experiment, we adjust a single parameter while maintaining the default values for the others and report the average results of 50 runs.

- *Number of Target Servers (n)*: The number of target servers that need f from the cloud server, varying from 16 to 48, with a default value of 32.
- *File Size (fs)*: The size of the file for distribution, ranging from 1 MB to 144 MB. In the experiments, we categorize the files into small files, sized [1 MB, 24 MB] and [25 MB, 48 MB], medium files, sized [49 MB, 72 MB] and [73 MB, 96 MB], as well as large files, sized [97 MB, 120 MB] and [121 MB, 144 MB]. The default range is [25 MB, 48 MB].
- *Number of Faulty Servers (sn)*: The number of entry servers experiencing runtime exceptions that cause transmission delays or failures due to dynamic workloads and fluctuating network conditions. It varies from 1 to 5 and is 3 by default. In the experiments, when a server experiences a transmission delay, it takes 20%–40% more time to complete data transmission. when a server fails, it stops all the transmissions going through and does not respond to any messages received.

Given a specific n , EdgeHydra employs a (k, m) EC code to encode files, where $m = 3$ and $k = n - m$. Please note that EdgeHydra does not require a specific block size; it can be determined based on application-specific factors such as the

TABLE III
PERFORMANCE COMPARISON
($n = 32, S_n = 3, F_s = [25MB, 48MB], M = 3$)

Metric	DataSync	Gossip	EDD-A	EdgeDis	EdgeHydra
Time (Seconds)	5.27	20.76	15.04	1.33	0.66
Cost (Units)	1,123.99	519.08	174.05	159.83	160.31

number of entry edge servers and the file size. In the experiments, it can be calculated based on k and fs , i.e., fs/k .

B. Experimental Results

1) *Overall Performance*: Table III compares the overall performance of all investigated approaches under default parameter settings. Compared with Gossip and EDD-A, DataSync, as the conventional approach, requires significantly less time to complete the EDD process, with a reduction of 74.61% and 64.96%, respectively. However, direct transmissions of multiple file replicas from the cloud server to edge servers increase the backhaul network traffic notably, which costs much more dearly than edge-to-edge transmissions. As a result, DataSync costs the most to distribute f , as demonstrated in Table III. Specifically, DataSync's traffic costs are 2.17x that of Gossip and 6.46x that of EDD-A. Gossip minimizes backhaul traffic effectively but takes the most time to complete data distribution. In the meantime, its gossiping mechanism incurs substantial communication overheads. Combining the strengths of DataSync and Gossip, EDD-A reduces the distribution cost remarkably, cutting it by 84.51% compared to DataSync. EDD-A also outperforms Gossip significantly, accelerating the distribution process by 27.53% and reducing the distribution cost by 66.47%. These results validate the advantage of leveraging edge-to-edge data transmissions for EDD.

EdgeDis, the state-of-the-art EDD approach, partitions f into blocks for parallel transmissions through multiple entry servers. Taking this advantage, it gains a profound reduction in distribution time. Table III demonstrates that, compared to DataSync, Gossip, and EDD-A, EdgeDis reduces the distribution time by 74.84%, 93.61%, and 91.18%, respectively. In the meantime, it manages to keep the distribution cost at a low level, about 85.78% lower than DataSync on average.

Compared to EdgeDis, EdgeHydra cuts the distribution time further by an average of 50.54%, as shown in Table III. This comes from its ability to tolerate faulty servers that slow down EdgeDis. In EdgeDis, the distribution process completes once all the data blocks are transmitted to all the target servers. In this experiment, EdgeDis has to retransmit data blocks impacted by faulty servers. These data blocks slow down EdgeDis. In contrast, EdgeHydra encodes f into coded blocks for parallel transmission, allowing target servers to reconstruct f with only a fraction of these coded blocks (Section III-B). Coded blocks impacted by faulty servers do not necessarily slow down EdgeHydra. However, due to the nature of erasure coding, EdgeHydra incurs redundant data transmissions. Nonetheless, the experimental results indicate that the traffic cost incurred by EdgeHydra is only 0.30% higher than EdgeDis. This comes from the superior ability of EdgeHydra to accommodate faulty servers

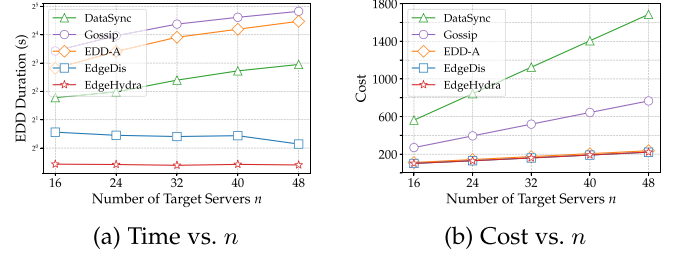


Fig. 6. Impact of number of target servers n .

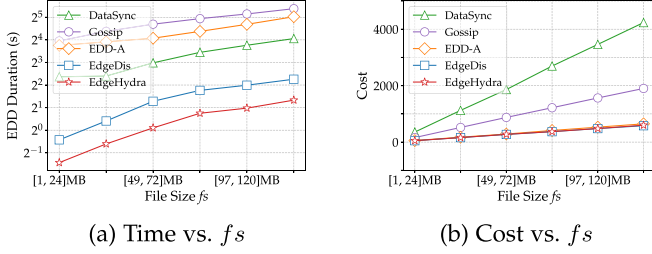
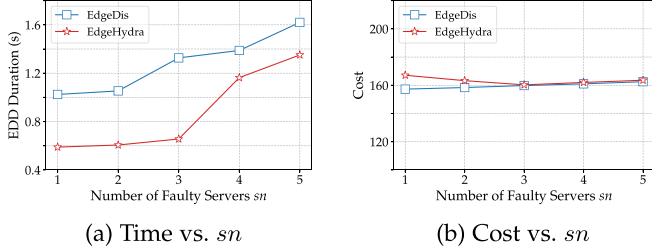
during the EDD process. It avoids most exception-triggered data re-transmissions that delay data distribution enormously under EdgeDis.

2) *Performance Versus Number of Target Servers*: Fig. 6 compares the distribution time and cost required by different approaches to complete the EDD process for different numbers of target servers (n). EdgeHydra is the clear winner in terms of distribution time and is very close to EdgeDis in terms of distribution cost.

As illustrated in Fig. 6(a), when n increases from 16 to 48, the average time taken by traditional methods such as DataSync, Gossip, and EDD-A to distribute data increases, whereas for EdgeDis and EdgeHydra, it decreases. A large n increases the bandwidth pressure on DataSync, Gossip, and EDD-A, taking them more time to complete data distribution to target servers. However, EdgeDis and EdgeHydra manage to take advantage of the increase in n because they can utilize more entry servers for parallel data transmissions, which accelerates the EDD process. Across all the different scenarios, EdgeHydra reduces the distribution time on average by 87.66%, 96.67%, 95.45%, and 49.72%, compared to DataSync, Gossip, EDD-A, and EdgeDis, respectively. It is the clear winner in handling large-scale data distribution.

Fig. 6(b) shows that all the EDD approaches have to experience cost increases as n increases. It comes from the greater volume of data transmitted over the network. On average, EdgeHydra incurs 85.66%, 68.92%, and 7.40% fewer distribution costs than DataSync, Gossip, and EDD-A, respectively, and only 0.40% more than EdgeDis. Notably, the cost increases for all approaches by a larger n are significantly lower than that for DataSync. This highlights the substantial economic benefits of leveraging edge-to-edge transmissions during the EDD process.

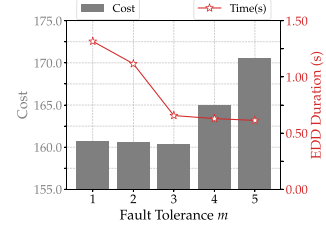
3) *Performance Versus File Size*: Fig. 7 compares the time different approaches take to distribute files of different sizes and the costs incurred. We can see that, EdgeHydra always takes less time to complete the distribution, despite the file size. In addition, its associated costs are very close to EdgeDis, much lower than the other approaches. Fig. 7(a) illustrates that the time required to complete the EDD process extends with an increase in file size. In the meantime, the impact of faulty servers becomes more significant. We can see that EdgeHydra can easily handle the distribution of large files. Across all file sizes, EdgeHydra is the fastest approach, saving an average of 86.09%, 95.14%, 93.36%, and 50.38% in distribution time compared to DataSync, Gossip,

Fig. 7. Impact of file size f_s .Fig. 8. Impact of number of faulty servers sn .

EDD-A, and EdgeDis, respectively. When the file size increases, the distribution costs more for all the approaches, unsurprisingly. This can be seen in Fig. 7(b). EdgeHydra always incurs the second lowest distribution cost of all, seconding to EdgeDis with a 1.28% extra on average. Compared with DataSync, Gossip, and EDD-A, EdgeHydra is much more economical, saving 85.89%, 68.94%, and 7.95%, respectively, compared with their distribution costs.

4) *Performance Versus Number of Faulty Servers*: As mentioned in Section I, EdgeDis benefits significantly from parallel data transmissions, but is susceptible to potential runtime exceptions in the volatile edge computing environment. EdgeHydra tolerates such faults by transmitting coded blocks instead of data blocks. By varying the number of faulty servers, we can analyze EdgeHydra's ability to complete EDD distribution in the presence of faulty servers. Fig. 8 illustrates the impact of the increase in the number of faulty servers sn from 1 to 5 on the performance of EdgeHydra and EdgeDis, with all other parameters held constant at their default values.

Fig. 8(a) reveals that when sn triples from 1 to 3, EdgeDis's distribution time escalates by 29.51% from 1.02 s to 1.33 s. In contrast, EdgeHydra's distribution time increases by only 11.41%. As expected, EdgeHydra can properly tolerate 3 faulty servers at the same time during the EDD process with $m = 3$. As sn goes beyond 3, it takes EdgeHydra more time to complete the EDD process, but consistently less than EdgeDis. Take $ec = 5$ for example. EdgeHydra is 16.60% faster than EdgeDis on average. This observation indicates that even when the number of faulty servers exceeds EdgeHydra's maximum fault tolerance capability, EdgeHydra can still complete the EDD process with the least extra time of all the EDD approaches. In this way, EdgeHydra can encode the files for distribution with a more

Fig. 9. Impact of fault tolerance m .

fault-tolerant EC coding, to boost its fault-tolerance capability before runtime exceptions get worse.

Fig. 8(b) shows that as sn increases, EdgeDis's distribution cost rises, whereas EdgeHydra's decreases first before increasing. When $sn \leq m = 3$, some coded blocks are not transmitted through faulty servers into the system and EdgeHydra does not have to re-transmit these coded blocks from the cloud. Its distribution cost decreases accordingly when sn increases from 1 to 3. Across all the cases, EdgeHydra's distribution cost is at most 6.25% higher than EdgeDis. This tells us EdgeHydra enables fault tolerance with minimal extra distribution costs.

5) *Performance Versus Fault Tolerance*: As demonstrated above, EdgeHydra achieves superior performance in various EDD scenarios by tolerating faulty servers with EC-based data transmissions. In the previous experiments, EdgeHydra utilized a (k, m) erasure coding scheme, allowing a target server to reconstruct file f with any k of the $k + m$ coded blocks and tolerate faulty servers that impact m coded blocks. However, while erasure coding facilitates fault tolerance, it also leads to redundant data transmissions. This increases the distribution overheads. To evaluate these overheads, we conduct an experiment where EdgeHydra employs different m values to encode files for distribution. Fig. 9 demonstrates the corresponding distribution time and cost.

We can see that as m increases from 1 to 3, the distribution time decreases markedly, falling from 1.32 s to 0.66 s by 50.19%. Notably, when m increases further from 3 to 5, the distribution time decreases by only 6.60%, much less than the decrease when m increases from 1 to 3. We looked into the experiment and found the reason. With a (k, m) EC coding, EdgeHydra can mitigate the impact of the m slowest servers on the EDD process. When the m does not exceed the number of faulty servers, each increase in m by 1 allows EdgeHydra to mitigate the impact of one extra faulty server. Thus, when m surpasses the total number of faulty servers, i.e., 3 in the experiment, a further increase in m yields no additional benefit in fault tolerance.

With an increased m , EdgeHydra may also incur more distribution costs. Fig. 9 demonstrates that when m increases from 3 to 5, its distribution cost rises by 6.37%. An interesting observation here is that the distribution cost increases unexpectedly when m decreases from 3 to 1. This occurs because if m falls below the number of faulty servers, the cloud has to retransmit impacted coded blocks, which incurs extra costs.

This experiment concludes that m needs to be specified domain-specifically for EdgeHydra to strike a proper balance

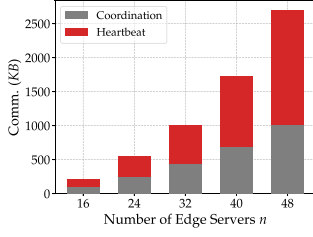


Fig. 10. Extra overheads.

between fault tolerance and distribution overheads. A rule of thumb is that, when the network conditions are stable and edge server failures are rare, a small m will allow EdgeHydra to accommodate occasional slow or failed entry servers without incurring significant extra traffic overhead. When network conditions fluctuate significantly, a large m will allow EdgeHydra to minimize the impact of slow entry servers at the price of extra traffic overhead incurred by the transmission of additional blocks. A large m will also allow EdgeHydra to minimize the impact of faulty entry servers.

6) *Extra Overheads Examination*: As introduced in Section III-D, under EdgeHydra, edge servers broadcast heartbeats in the system periodically to notify other target servers of the coded blocks it has received. This incurs traffic overheads. Moreover, coordinating the EDD process requires the servers to exchange messages, e.g., requests, responses, and IDs, which also incurs traffic overheads. Fig. 10 demonstrates these traffic overheads in edge caching systems of different sizes, including heartbeat overheads and coordination overheads. As the number of target servers n increases, the heartbeat overheads incurred by EdgeHydra's block supplement mechanism (Section III-D) increase. This is because there are more servers broadcasting heartbeats in the system, which translates to more heartbeat overheads. Overall, the communication overhead introduced by EdgeHydra remains minimal across all scenarios. Even with $n = 48$, the traffic overhead incurred is only 2,699.01KB in total, of which the heartbeat overhead is only 1689.89KB. This confirms that heartbeat broadcasts impose a minimal burden on edge servers and do not result in broadcast storms.

7) *Ablation Study*: As introduced in Section III-D, EdgeHydra supplies missing coded blocks to target servers during block supplement. This takes time. To fully understand the effect of EdgeHydra's leaderless mechanism (Section III-D) for supplying missing coded blocks, we conduct an ablation study where EdgeHydra leaderless block supplement mechanism is replaced with EdgeDis's coordinator mechanism. We evaluate their effectiveness by measuring the duration required to supply missing coded blocks, and investigate how the number of target servers (n) and file size (fs) influence the data supplement process.

Fig. 11 illustrates the experimental results. Fig. 11(a) demonstrates that compared with the leader-based mechanism, EdgeHydra's leaderless mechanism requires less time to supply missing coded blocks to target servers, achieving an average time savings of 33.44%. In addition, the increase in n has a greater impact on the leader-based mechanism. When it increases from

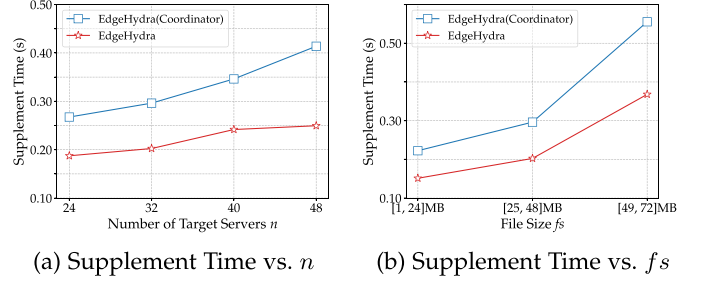


Fig. 11. Ablation study.

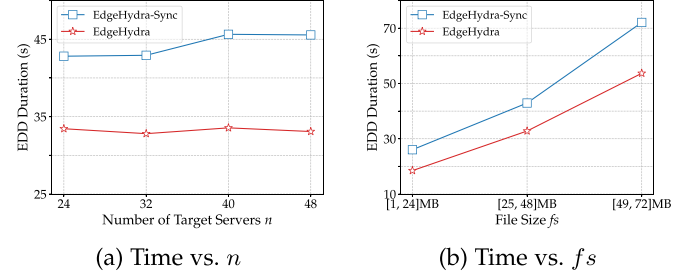


Fig. 12. Multi-file distribution.

24 to 48, the leader-based mechanism takes 54.70% more time to supply all the missing coded blocks, whereas the increase is only 33.16% for EdgeHydra's leaderless mechanism. Fig. 11(b) shows that across the entire file size range, EdgeHydra's leaderless mechanism saves an average of 32.82% in block supplement time compared to the leader-based mechanism. This experiment validates that EdgeHydra's leaderless block supplement mechanism can indeed accelerate the supplement by distributing the workloads across multiple edge servers, rather than centralizing it on a single coordinator server potentially subject to computation computation and network resource constraints.

8) *Performance of Multi-File Distribution*: As introduced in Section III-B, EdgeHydra allows asynchronous distributions of files between Stage 1 and Stages 2–4. This is another advantage of EdgeHydra. To evaluate this advantage, we compare the performance of EdgeHydra against EdgeHydra-Sync, a variant of EdgeHydra that distributes files one after another, measured by the time taken to distribute a total of 50 files. Fig. 12(a) shows the results. Across different numbers of target servers (n), EdgeHydra requires 25.58% less time on average to distribute all 50 files than EdgeHydra-Sync. Fig. 12(b) demonstrates that across the entire range of file sizes (fs), EdgeHydra takes 24.90% less time to distribute all 50 files on average. This experiment confirms that asynchronous file distribution allows EdgeHydra to distribute files more efficiently.

9) *Discussion*: The experimental results demonstrate that, by transmitting blocks erasure-coded from the original file(s), EdgeHydra can effectively tolerate transmission failures during edge data distribution. However, the traffic overhead incurred is not negligible. Thus, in real-world applications, EdgeHydra can make a profound contribution to ensuring high quality of service for users that demand prompt data delivery, especially when high block redundancy is needed at peak hours to cope with significant network fluctuations and/or frequent node failures.

V. RELATED WORK

Edge computing, empowered by 5G communication technologies, extends cloud computing by delivering computing and storage capabilities directly at the network's edge [35], [36]. Popular data can be cached on edge servers proximate to end-users by app vendors, enabling superfast data retrieval, which is key to many real-time application [37], [38], [39].

In recent years, the problem of edge data caching (EDC) has obtained extensive research attention from academic and industrial communities. Researchers are proposing and exploring innovative ideas and technologies for effective and efficient caching strategies for edge caching systems. A substantial body of research focuses on enhancing the cost-effectiveness of EDC from the perspective of app providers through collaborative data caching across multiple servers in the edge caching systems [17], [20], [38], [40]. Researchers have also identified a series of new issues that arise with the popularity of EDC, e.g., workload fairness [14], data integrity [18], [28], and demand dynamics [19], [38], [41].

To implement an EDC strategy dictating the allocation of specific data to various edge servers, the cloud must distribute data to the edge servers in the edge caching system. Unlike conventional caching systems, the physical distance between cloud servers and edge servers contributes significantly to the total duration required to complete edge data distribution (EDD) [6]. In addition, app vendors are charged for every MB of data transmitted during the EDD. In the edge computing environment, users' data demands often vary substantially both spatially [42], [43] and temporally [44], [45], [46]. Inefficient EDD strategies can under the ability of edge caching systems to accommodate users' data demands and incur excessive transmission costs over time [24].

A straightforward EDD solution is to employ Amazon's DataSync service [23] to transfer data from cloud servers directly to all the target servers. However, this approach can generate excessive traffic on the backhaul network, resulting in prohibitively high transmission costs for app vendors, as demonstrated in Section IV. Alternatively, an EDD approach can transmit data to one of the servers in the edge caching system as the entry server, and spread the data in the system to other target servers based on a consensus protocol such as Gossip [25] or Raft [34]. While this method reduces backhaul traffic costs significantly, it inevitably increases the time required for the EDD process to complete due to massive (and often redundant) gossiping in the system. This was also confirmed experimentally in Section IV. An approach named EDD-A was proposed by Xia et al. that considers both the cost and time factors [24]. Given a set of target servers, it generates a Steiner tree that spans from the cloud server to all target servers, and transmits data along the paths in the tree [24]. The key limitation of this approach is its vulnerability to runtime exceptions. A runtime exception in any of the nodes and edges in the tree-based distribution paths will delay or even terminate the entire EDD procedure. The delays and costs caused by such exceptions are unacceptable, as shown above in Section IV.

Very recently, Li et al. proposed EdgeDis to accelerate EDD through parallel data transmissions [21]. Under the EdgeDis scheme, a file f is not transmitted as a whole. Instead, it is

sliced into some data blocks to be distributed to target servers simultaneously through multiple entry servers. It reduces the backhaul traffic and achieves profound speedups compared with previous EDD approaches. However, it overlooked the impact on the EDD process when entry servers fail to transmit successfully and promptly, as illustrated and analyzed in Section II.

EdgeHydra aims to overcome the limitations of existing EDD approaches discussed above with the innovative utilization of erasure coding during the EDD process. It is capable of ensuring economic EDD with improved robustness against exceptions during the EDD process.

VI. CONCLUSION

In this paper, we studied the edge data distribution (EDD) problem and proposed a new erasure coding-based EDD scheme named EdgeHydra. With erasure coding, it encodes files into coded blocks for parallel transmissions, accelerating the distribution process and in the meantime ensuring robustness against runtime exceptions. With a new decentralized block supplement mechanism, EdgeHydra guarantees the completion of EDD with balanced transmission workloads. Experimental results indicate that EdgeHydra achieves superior performance in various EDD scenarios, outperforming the state-of-the-art by up to 50.54% in distribution duration.

REFERENCES

- [1] A. Osseiran et al., "The foundation of the mobile and wireless communications system for 2020 and beyond: Challenges, enablers and technology solutions," in *Proc. IEEE Veh. Technol. Conf.*, 2013, pp. 1–5.
- [2] Z. Xu et al., "Age-aware data selection and aggregator placement for timely federated continual learning in mobile edge computing," *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 466–480, Feb. 2024.
- [3] R. Bhardwaj et al., "Ekya: Continuous learning of video analytics models on edge compute servers," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 119–135.
- [4] X. Hou and S. Dey, "Motion prediction and pre-rendering at the edge to enable ultra-low latency mobile 6DoF experiences," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 1674–1690, 2020.
- [5] F. Nawab, D. Agrawal, and A. El Abbadi, "DPaxos: Managing data closer to users for low-latency and mobile applications," in *Proc. Int. Conf. Manage. Data*, 2018, pp. 1221–1236.
- [6] Amazon, "AWS wavelength for media & entertainment," 2021. [Online]. Available: <https://d1.awsstatic.com/Wavelengthyear2020/AWS-Wavelength-for-Media-Entertainment-SolutionBrief-Feb2021-Final.pdf>
- [7] Z. Xu, Y. Fu, Q. Xia, and H. Li, "Enabling age-aware Big Data analytics in serverless edge clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [8] E. Schurman and J. Brutlag, "The user and business impact of server delays, additional bytes, and HTTP chunking in web search," *Velocity Web Perform. Operations Conf.*, Oreilly, 2009.
- [9] S.-C. Lin et al., "The architectural implications of autonomous driving: Constraints and acceleration," in *Proc. 23rd Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2018, pp. 751–766.
- [10] S. Ma, S. Guo, K. Wang, W. Jia, and M. Guo, "A cyclic game for service-oriented resource allocation in edge computing," *IEEE Trans. Serv. Comput.*, vol. 13, no. 4, pp. 723–734, Jul./Aug. 2020.
- [11] Z. Xu et al., "Collaborate or separate? distributed service caching in mobile edge clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2066–2075.
- [12] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
- [13] X. Xia, F. Chen, Q. He, J. Grundy, M. Abdelrazek, and H. Jin, "Online collaborative data caching in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 281–294, Feb. 2021.

- [14] J. Zhou, F. Chen, Q. He, X. Xia, R. Wang, and Y. Xiang, "Data caching optimization with fairness in mobile edge computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1750–1762, May/Jun. 2023.
- [15] R. Luo, H. Jin, Q. He, S. Wu, and X. Xia, "Enabling balanced data deduplication in mobile edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1420–1431, May 2023.
- [16] X. Xia et al., "Formulating cost-effective data distribution strategies online for edge cache systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4270–4281, Dec. 2022.
- [17] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.
- [18] B. Li et al., "Cooperative assurance of cache data integrity for mobile edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4648–4662, 2021.
- [19] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 207–215.
- [20] J. Peng et al., "MagNet: Cooperative edge caching by automatic content congregating," in *Proc. ACM Web Conf.*, 2022, pp. 3280–3288.
- [21] B. Li, Q. He, F. Chen, L. Lyu, A. Bouguettaya, and Y. Yang, "EdgeDis: Enabling fast, economical, and reliable data dissemination for mobile edge computing," *IEEE Trans. Serv. Comput.*, vol. 17, no. 4, pp. 1504–1518, Jul./Aug. 2024.
- [22] Q. Xia, Z. Xu, W. Liang, S. Yu, S. Guo, and A. Y. Zomaya, "Efficient data placement and replication for QoS-aware approximate query evaluation of Big Data analytics," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 12, pp. 2677–2691, Dec. 2019.
- [23] Amazon, "AWS DataSync," 2018, Accessed: Mar. 20, 2022. [Online]. Available: <https://aws.amazon.com/datasync/>
- [24] X. Xia, F. Chen, Q. He, J. Grundy, M. Abdelrazek, and H. Jin, "Cost-effective app data distribution in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 31–44, Jan. 2021.
- [25] S. Boyd, A. Ghosh, B. Prabhakar, and D. Shah, "Randomized gossip algorithms," *IEEE Trans. Inf. Theory*, vol. 52, no. 6, pp. 2508–2530, Jun. 2006.
- [26] L. Yuan et al., "CoopEdge: A decentralized blockchain-based platform for cooperative edge computing," in *Proc. Web Conf.*, 2021, pp. 2245–2257.
- [27] B. Li, Q. He, F. Chen, H. Jin, Y. Xiang, and Y. Yang, "Auditing cache data integrity in the edge computing environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1210–1223, May 2021.
- [28] B. Li, Q. He, L. Yuan, F. Chen, L. Lyu, and Y. Yang, "EdgeWatch: Collaborative investigation of data integrity at the edge based on blockchain," in *Proc. 28th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 3208–3218.
- [29] Q. He et al., "A game-theoretical approach for mitigating edge DDoS attack," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2333–2348, Jul./Aug. 2022.
- [30] C. Huang et al., "Erasure coding in windows azure storage," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 15–26.
- [31] K. V. Rashmi, N. B. Shah, D. Gu, H. Kuang, D. Borthakur, and K. Ramchandran, "A hitchhiker's guide to fast and efficient data reconstruction in erasure-coded data centers," in *Proc. ACM SIGCOMM Conf.*, 2014, pp. 331–342.
- [32] J. Yang, A. Sabnis, D. S. Berger, K. Rashmi, and R. K. Sitaraman, "C2DN: How to harness erasure codes at the edge for efficient content delivery," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 1159–1177.
- [33] H. Jin, R. Luo, Q. He, S. Wu, Z. Zeng, and X. Xia, "Cost-effective data placement in edge storage systems with erasure code," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1039–1050, Mar./Apr. 2023.
- [34] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 305–319.
- [35] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop multi-task partial computation offloading in collaborative edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1133–1145, May 2021.
- [36] L. He, F. Li, H. Xu, W. Xia, X. Zhang, and X. Tao, "Blockchain-based vehicular edge computing networks: The communication perspective," *Sci. China Inf. Sci.*, vol. 66, no. 7, 2023, Art. no. 172301.
- [37] S. Li and T. Lan, "Hotdedup: Managing hot data storage at network edge through optimal distributed deduplication," in *Proc. Conf. Comput. Commun.*, 2020, pp. 247–256.
- [38] X. Xia et al., "Data, user and power allocations for caching in multi-access edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 5, pp. 1144–1155, May 2022.
- [39] X. Xia et al., "Budgeted data caching based on k-median in mobile edge computing," in *Proc. 27th IEEE Int. Conf. Web Serv.*, 2020, pp. 197–206.
- [40] T. X. Tran and D. Pompili, "Adaptive bitrate video caching and processing in mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 1965–1978, Sep. 2019.
- [41] T. Zhao, I.-H. Hou, S. Wang, and K. Chan, "Red/LeD: An asymptotically optimal and scalable online algorithm for service caching at the edge," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 8, pp. 1857–1870, Aug. 2018.
- [42] P. Yang, N. Zhang, S. Zhang, L. Yu, J. Zhang, and X. S. Shen, "Content popularity prediction towards location-aware mobile edge caching," *IEEE Trans. Multimedia*, vol. 21, no. 4, pp. 915–929, Apr. 2019.
- [43] Y. Jiang, M. Ma, M. Bennis, F.-C. Zheng, and X. You, "User preference learning-based edge caching for fog radio access network," *IEEE Trans. Commun.*, vol. 67, no. 2, pp. 1268–1283, Feb. 2019.
- [44] X. Ma, S. Wang, S. Zhang, P. Yang, C. Lin, and X. S. Shen, "Cost-efficient resource provisioning for dynamic requests in cloud assisted mobile edge computing," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 968–980, Third Quarter 2021.
- [45] S. M. Azimi, O. Simeone, A. Sengupta, and R. Tandon, "Online edge caching and wireless delivery in fog-aided networks with dynamic content popularity," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 6, pp. 1189–1202, Jun. 2018.
- [46] J. Gao, S. Zhang, L. Zhao, and X. S. Shen, "The design of dynamic probabilistic caching with time-varying content popularity," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.



Qiang He (Senior Member, IEEE) received the first PhD degree from the Swinburne University of Technology, Australia, in 2009 and the second PhD degree from the Huazhong University of Science and Technology, China, in 2010. He is currently a full professor. His research interests include edge computing, cloud computing and service computing.



Guobiao Zhang received the bachelor's degree from Northeastern University, China, in 2023. He is currently working toward the master's degree with the Huazhong University of Science and Technology (HUST), China. His research interests include edge computing, parallel and distributed computing, and cloud computing.



Jiawei Wang received the bachelor's degree in computer science and technology with the Huazhong University of Science and Technology (HUST), China, in 2024. He is currently working toward the master's degree with HUST. His research interests lie in the fields of edge computing and software engineering.



Ruikun Luo received the bachelor's degree from Dalian Maritime University, China, in 2018, and the PhD degree from the Huazhong University of Science and Technology (HUST), China, in 2023. Now he is a postdoctoral research fellow with HUST. His research interests include edge computing, parallel and distributed computing, and network storage.



Xiaohai Dai (Member, IEEE) received the PhD degree from the School of Computer Science and Technology, Huazhong University of Science and Technology (HUST), Wuhan, China, in 2021. He is currently a postdoctoral research fellow with the School of Computer Science and Technology, HUST. His current research interests include blockchain and distributed systems. His awards include the Outstanding Creative Award in the 2018 FISCO BCOS Blockchain Application Contest and Top Ten in Fin-Techathon 2019.



Yuchong Hu received the PhD degree in computer software and theory from the University of Science and Technology of China, in 2010. He is now a professor with the School of Computer Science and Technology, Huazhong University of Science and Technology. His research interests are dependability in distributed systems, including cloud computing, neuromorphic computing, and data centers.



Feifei Chen (Senior Member, IEEE) received the PhD degree from the Swinburne University of Technology, Australia, in 2015. She is a senior lecturer with Deakin University, Australia. Her research interests include software engineering, edge computing, cloud computing, and green computing.



Hai Jin (Fellow, IEEE) received the PhD degree in computer engineering from the Huazhong University of Science and Technology, in 1994. He is a chair professor of computer science and engineering with the Huazhong University of Science and Technology (HUST), China. In 1996, he was awarded a German Academic Exchange Service fellowship to visit the Technical University of Chemnitz in Germany. He worked with the University of Hong Kong between 1998 and 2000, and as a visiting scholar with the University of Southern California between 1999 and 2000. He was awarded the Excellent Youth Award from the National Science Foundation of China in 2001. Jin is a fellow of CCF, and a life member of the ACM. His research interests include computer architecture, parallel and distributed computing, Big Data processing, data storage, and system security.



Yun Yang (Senior Member, IEEE) received the PhD degree in computer science from the University of Queensland, Australia, in 1992. He is currently a full professor with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia. His research interests include software technologies, cloud and edge computing, workflow systems, and service computing. He was an associate editor for *IEEE Transactions on Parallel and Distributed Systems* from 2018 to 2022.